

1. Qué es el proyecto (contexto que se deduce del código)

Es un **simulador de logística de última milla urbana** (proyecto GLIMS). Compara cinco modelos de reparto (M1–M5) barrio a barrio en Barcelona, Madrid y Valencia, calculando para cada uno kilómetros recorridos, número de viajes, emisiones de CO₂ y coste total en euros. Hay dos generaciones del cálculo:

- Una **versión "geométrica"** basada en distancias en línea recta (Haversine): Barcelona.ipynb y el módulo simulador.py (no visible).
- Una **versión "realista"** basada en la red viaria real de OpenStreetMap con OSMnx: simulador_osmnx.py.

2. Cómo se relacionan los archivos

El flujo de datos y dependencias es este:

Excel crudos (Points B2C, CC)

|



preparar_datos.py —► genera CSVs en /data/<ciudad>/

|

(puntos_b2c.csv, centros_cc.csv,

|

limites_barrios.csv, parametros_modelos.csv)



simulador_osmnx.py —► lee esos CSVs + descarga red OSM

|

simula M1–M5 → DataFrame de resultados

|

(importa calcular_haversine de ► simulador.py [no visible])



pruebas_osmnx.ipynb —► llama a simular_ciudad(), guarda /results/ y hace gráficas

(también importa el módulo simulador)

Barcelona.ipynb —► prototipo autónomo (versión Haversine, no depende de nada)

consultas.ipynb —► exploración suelta de barrios con OSMnx (independiente)

En resumen: preparar_datos.py produce los datos, simulador_osmnx.py es el motor de cálculo, pruebas_osmnx.ipynb es el "cuaderno de ejecución y análisis", y Barcelona.ipynb / consultas.ipynb son material más antiguo o exploratorio.

3. Archivo por archivo

preparar_datos.py — ETL de datos crudos a CSVs

Toma los dos Excel de origen (Points B2C, CC), los limpia y los reparte en carpetas por ciudad. También materializa a CSV los límites de barrios y los parámetros de los modelos, que están definidos como constantes en el propio archivo.

Constantes clave: RAW_DATA, ARCHIVO_PUNTOS, ARCHIVO_CC, CARPETA_SALIDA (rutas absolutas de Windows), CIUDADES (mapea el nombre de la pestaña del Excel a un *slug*), LIMITES_BARRIOS (bounding boxes por barrio) y PARAMETROS_MODELOS (coste/km, coste/hora, velocidad, CO₂, capacidad... de cada modelo).

Funciones:

- **normalizar_texto(texto)**: limpia una cadena (quita el carácter invisible \u200b, normaliza Unicode con NFKC, colapsa espacios). Es la pieza que permite casar nombres de pestaña aunque traigan espacios "raros".
- **encontrar_hoja(excel, nombre_objetivo)**: busca en un ExcelFile la hoja cuyo nombre normalizado coincide con el buscado; lanza ValueError si no la encuentra. Resuelve el problema de que las pestañas del Excel llevan espacios dobles/invisibles.
- **limpiar_dataframe(df)**: elimina columnas Unnamed y columnas totalmente vacías, normaliza los nombres de columna, convierte Latitude/Longitude a numérico (cambiando comas por puntos) y descarta filas sin coordenadas.
- **validar_archivos()**: comprueba que existen los dos Excel de entrada; si no, lanza FileNotFoundError.
- **exportar_hojas_por_ciudad(archivo_excel, salida, nombre_csv)**: recorre CIUDADES, localiza la hoja de cada ciudad, la lee con header=1 (salta la primera fila decorativa), la limpia y la guarda como CSV en salida/<ciudad>.
- **exportar_limites_barrios(salida)**: vuelca LIMITES_BARRIOS a un limites_barrios.csv por ciudad.
- **exportar_parametros(salida)**: vuelca PARAMETROS_MODELOS a parametros_modelos.csv (uno global).
- **preparar_datos()**: orquestador. Valida, crea la carpeta de salida y llama a los cuatro exportadores en orden. Es el punto de entrada (if __name__ == "__main__").

simulador_osmnx.py — motor de simulación sobre red viaria real

Es el archivo central. Carga los CSVs, descarga la red de calles de la ciudad con OSMnx, asocia cada cliente y centro al nodo más cercano de la red, y para cada barrio calcula los cinco modelos usando distancias por carretera (Dijkstra / A*).

Constantes: BASE_DIR (dos niveles por encima del archivo), CIUDAD_ACTIVA, BARRIO_ACTIVO, CARPETA_DATA, CARPETA_RESULTADOS.

Funciones:

- **cargar_datos_ciudad(ciudad)**: lee los cuatro CSVs (puntos, centros, límites, parámetros) y los devuelve.
- **obtener_parametros(parametros)**: convierte el DataFrame de parámetros en un dict indexado por modelo, para acceso cómodo tipo parametros["FURGONETA_CONV"].

- **cargar_redes(ciudad)**: descarga con `ox.graph_from_place` la red tipo drive de la ciudad e imprime nº de nodos/aristas.
- **construir_subgrafo_barrio(G_drive, barrio, buffer=0.01)**: recorta el grafo a la bounding box del barrio (más un margen) con `ox.truncate.truncate_graph_bbox`.
- **asignar_nodos(puntos, centros, G_drive)**: asigna a cada cliente y centro el nodo más cercano de la red (`ox.distance.nearest_nodes`), guardándolo en la columna `node_drive`.
- **filtrar_puntos_barrio(puntos, barrio)**: filtra los puntos que caen dentro de la bounding box del barrio.
- **preparar_barrio(puntos_barrio, G_drive)**: calcula el centroide (media de lat/lon) del barrio, su nodo más cercano y la lista de nodos de los clientes; lo devuelve como dict `info_barrio`.
- **seleccionar_centro_logistico(centros, nodo_centroide, G_drive)**: con un Dijkstra desde el centroide, calcula la distancia por carretera a cada centro y devuelve el más cercano.
- **construir_matriz_distancias(G, lista_nodos)**: construye la matriz N×N de distancias mínimas (en km) entre una lista de nodos, lanzando un Dijkstra desde cada origen. Devuelve la matriz y un `mapa_nodos` (nodo → índice).
- **obtener_matrices_barrio(info_barrio, G_drive)**: envuelve la anterior para el conjunto centro + clientes del barrio.
- **tsp_vecino_mas_cercano(nodo_inicio, clientes, matriz, mapa_nodos)**: heurística TSP del vecino más cercano; recorre todos los clientes eligiendo siempre el más próximo y suma el regreso al inicio. Devuelve los km de ruta (reparto en furgoneta/bici).
- **calcular_distancia_radial(nodo_origen, clientes, matriz, mapa_nodos)**: suma los trayectos ida-y-vuelta independientes desde el origen a cada cliente (modelo "radial", usado en reparto a pie y recogida por el cliente).
- **simular_barrio(...)**: el corazón. Prepara el barrio, elige centro logístico, construye la matriz, calcula la distancia troncal CC→barrio (con `nx.astar_path_length` y heurística Haversine), los km internos (TSP) y los radiales, y a partir de ahí calcula los cinco modelos M1–M5, devolviendo una lista de dicts (uno por modelo).
- **simular_ciudad(ciudad, barrio_activo=None)**: orquesta todo para una ciudad: carga datos y red, asigna nodos, filtra por barrio si se pide, y recorre los barrios llamando a `simular_barrio`. Devuelve un DataFrame con todos los resultados.
- Bloque `__main__`: imprime cabecera, ejecuta `simular_ciudad` con las constantes y guarda el CSV en `/results/`.

Los cinco modelos dentro de `simular_barrio`, en una frase cada uno: **M1** furgoneta de combustión desde el CC; **M2** furgoneta eléctrica (mismo esquema, sin emisiones directas); **M3** camión abastece un microhub y reparte en bici-carga (con coste fijo diario del hub); **M4** camión abastece un PUDO y un repartidor entrega a pie de forma radial; **M5** igual que M4 pero recoge el propio cliente (emisiones estimadas de sus desplazamientos).

Importado por `simulador_osmnx.py` (de él saca `calcular_haversine`) y por `pruebas_osmnx.ipynb` (`CARPETA_DATA`, `calcular_haversine`, y se recarga con `importlib.reload`). Por cómo se usa y por `Barcelona.ipynb`, es casi seguro **la versión Haversine del simulador**, con al menos `calcular_haversine`, la constante `CARPETA_DATA` y probablemente un `simular_kilometros_tsp/simular_ciudad` propios. **No puedo describir sus funciones con certeza porque no está subido**; si quieres que lo documente, súbelo.

`pruebas_osmnx.ipynb` — cuaderno de ejecución y análisis

No define lógica de negocio; es el *driver*. Importa `simular_ciudad` de `simulador_osmnx` y el módulo `simulador`, ejecuta la simulación para una ciudad/barrio, guarda los resultados en `/results/<ciudad>/` (un resumen y un CSV por barrio) y produce numerosas visualizaciones con `matplotlib/seaborn`: barras de coste y CO₂ por modelo, `heatmaps` barrio×modelo, tabla de "modelos ganadores" (mínimo coste por barrio), y un consolidado leyendo todos los CSVs de `/results/` para comparar coste medio, CO₂ medio y un `scatter` coste-vs-emisiones. Importa `sklearn` pero no llega a usarlo en las celdas.

`Barcelona.ipynb` — prototipo autónomo (versión Haversine)

Es una versión **anterior y completa en un solo notebook**: instala `openpyxl`, lee los Excel directamente, filtra el barrio, define `PARAMETROS`, y calcula M1–M5 con distancias Haversine. Define dos funciones locales: **`calcular_haversine(lat1, lon1, lat2, lon2)`** (distancia esférica entre dos puntos) y **`simular_kilometros_tsp(inicio_lat, inicio_lon, df_puntos_ruta)`** (mismo TSP del vecino más cercano pero sobre coordenadas, no sobre grafo). Es el ancestro directo de `simulador.py` y de `simulador_osmnx.py`.

`consultas.ipynb` — exploración suelta

Tres celdas independientes: descarga los barrios de Barcelona con `ox.features_from_place` y los dibuja. No forma parte del pipeline; es una consulta exploratoria de OSMnx.

4. Problemas y deuda técnica que conviene que sepas

Aunque solo pedías la explicación, marco lo relevante para el mantenimiento:

Triple duplicación de la fuente de verdad. Los bounding boxes de barrios y los `PARAMETROS` de los modelos están repetidos en `Barcelona.ipynb`, en `preparar_datos.py` y (con casi total seguridad) en `simulador.py`. La misma lógica M1–M5 vive duplicada en `Barcelona.ipynb` y en `simulador_osmnx.py`. Cualquier cambio de parámetros hay que hacerlo en varios sitios: es una fuente de bugs silenciosos.

Posible bug de grafos mezclados en `simular_barrio`. Los nodos de clientes y centro se calculan sobre el grafo completo `G_drive` (en `asignar_nodos/preparar_barrio`), pero la matriz de distancias se construye sobre el **subgrafo** `G_barrio` (`obtener_matrices_barrio(info_barrio, G_barrio)`). Si algún nodo de cliente queda fuera del recorte, `construir_matriz_distancias` hará un `single_source_dijkstra_path_length` sobre un nodo inexistente en `G_barrio` y **lanzará excepción**, o si sobrevive pero queda desconectado, la distancia será `inf`. Merece una comprobación.

Rutas absolutas y convenciones inconsistentes. `preparar_datos.py` escribe en rutas absolutas de Windows (`E:/UPV/Proyectos/GLIMS/data`), mientras que `simulador_osmnx.py` usa rutas

relativas (BASE_DIR/../data). Si esas dos rutas no apuntan al mismo sitio, el simulador no encontrará los CSVs. Además rompe la portabilidad a otra máquina.

Código muerto y de depuración en simulador_osmnx.py. La función `distancia_cc_barrio` y su llamada están comentadas; hay varios print de depuración en medio de `simular_barrio` (centroide, nodo, "Separación", el bloque OSM: / HAV: que compara una distancia concreta). Eso debería salir del flujo de producción o pasar a logging.

Rendimiento. `cargar_redes` descarga la red completa de la ciudad en cada ejecución (mitigado en parte por la caché de OSMnx). Y `construir_matriz_distancias` lanza un Dijkstra completo por cada nodo: para barrios con muchos clientes es $O(N \cdot (E + V \log V))$; con recorte al subgrafo es asumible, pero conviene tenerlo presente.

Imports sin uso: `sklearn` en `pruebas_osmnx.ipynb`; `numpy` importado en `simulador_osmnx.py` con la semilla `np.random.seed(42)` comentada (`numpy` sí se usa por `np.ceil/np.inf`, así que ese import está bien; la semilla comentada no).

5. Mejoras futuras razonables (sin sobreingeniería)

Centralizar parámetros y límites de barrios en una única fuente (los CSVs que ya genera `preparar_datos.py`) y que todo el mundo lea de ahí, eliminando las copias en notebooks. Unificar las dos versiones del simulador (Haversine y OSMnx) tras una interfaz común para no mantener la lógica M1–M5 por duplicado. Sacar las rutas a configuración relativa/variable de entorno. Y extraer el cálculo de cada modelo M1–M5 a funciones pequeñas con una única responsabilidad (hoy `simular_barrio` hace demasiado), lo que además lo haría testeable.

¿Quieres que empiece por alguno de estos puntos? Si subes `simulador.py` completo, te hago también su documentación y te confirmo el posible bug del subgrafo con el código delante.